

Lecture 4: CNNs and Regularisation
Class Activation Maps, Adversarial Attacks
CNNs on Blood Cells and Satellite Images
Finding T_c of the 2D Ising Model

Deep Learning for Scientific Discovery



Dr Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

Recap: MNIST, the Deep Learning "Hello World"

1 output, MSE

$\mathcal{L}_{\text{MSE}} = (\bar{y} - y)^2$

$y \in \{0, \ldots, 9\}$: true digit, $\bar{y}$: network output

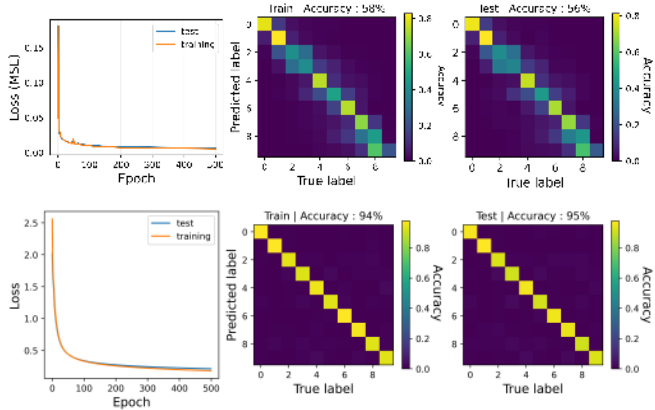
test accuracy 56%

10 outputs, cross-entropy

$\mathcal{L}_{\text{CE}} = -\log q(\text{true digit})$

$q(k)$: softmax output for digit k

test accuracy 95%



Losses per image, averaged over the training set; same hidden layer (21 sigmoid nodes). Confusion matrix: row = predicted, column = true digit. A single output orders the digits, so MSE errors lie next to the diagonal (3 taken for 2, 9 for 8).

the formulation of the problem matters as much as the network

1

Hello World #2: CIFAR-10 and Convolutional Networks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

CIFAR-10 Dataset



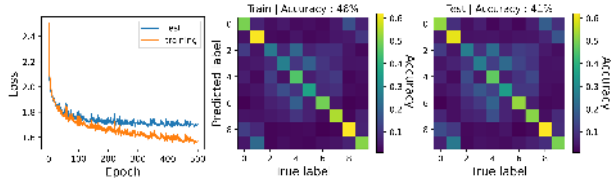
- 10 classes: aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
- 50 000 training, 10 000 test images
- 32×32 pixels in **3 colour channels** (R, G, B): each image is a $32 \times 32 \times 3$ tensor
- MNIST was $28 \times 28 \times 1$

A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (2009).

CIFAR-10: The Same Naive Network + Cross-Entropy

Exactly the MNIST recipe:

- **flatten** the image:
  $32 \cdot 32 \cdot 3 = 3072$ inputs
- one hidden layer, 21 sigmoid nodes
- 10 outputs, softmax, cross-entropy



$3072 \cdot 21 + 21 + 21 \cdot 10 + 10 = 64\,753$

parameters

- **Poor performance:** about 46% training, 41% test accuracy — the model *underfits*

What do we lack in our approach?

Flattening Loses Spatial Structure

Neighbouring pixels are correlated: an edge, a texture, an eye are *local* patterns of a few nearby pixels.

After flattening, the dense layer treats pixel #1 and pixel #2 as unrelated as pixel #1 and pixel #3000.

Shuffle all pixels with one fixed permutation: a dense network learns equally well — it never used the 2D layout.

What an image model should know

- **locality:** look at small neighbourhoods
- **weight sharing:** the same edge detector is useful everywhere in the image
- **translation:** a cat in the corner is still a cat

$\Rightarrow$ **capture spatial correlations** with *convolutions*.

Convolution: the Continuous Definition (1D)

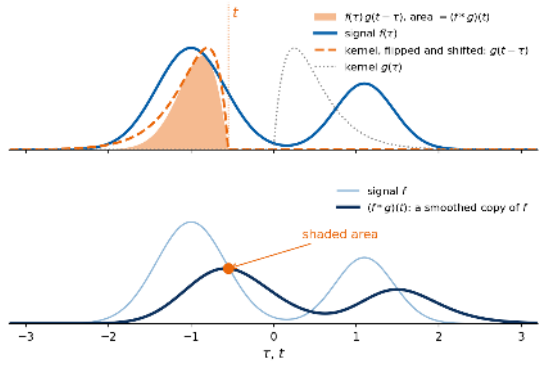
Two functions $f(\tau)$ (the signal) and $g(\tau)$ (the kernel):

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t - \tau)\, d\tau$$

For each output point t:

- **flip** the kernel, $g(-\tau)$
- **shift** it by t, $g(t - \tau)$
- **multiply** with $f(\tau)$
- **integrate**: the shaded area is $(f * g)(t)$

Moving t traces the whole curve $f * g$, here a **smoothed** copy of f.



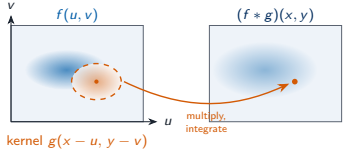
Convolution: the Continuous Definition (2D)

For functions of two variables, e.g. an image $f(x, y)$ and a kernel $g(x, y)$:

$$(f * g)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(u, v)\, g(x - u,\, y - v)\, du\, dv$$

the same operations as in 1D: flip g in both directions, shift it to (x, y), multiply with f, integrate over the plane; one number for each point (x, y)

g a 2D Gaussian $\Rightarrow$ $f * g$ is f **blurred**: every point replaced by a weighted average of its neighbourhood



the kernel, centred at (x, y), weights the image around that point; the integral gives the output there

Convolution theorem: in Fourier space a convolution is a product

$$\mathcal{F}[f](k) = \int f(t)\, e^{-ikt}\, dt, \qquad \mathcal{F}[f * g](k) = \mathcal{F}[f](k) \cdot \mathcal{F}[g](k), \qquad f * g = \mathcal{F}^{-1}\big[\mathcal{F}[f] \cdot \mathcal{F}[g]\big]$$

in 2D the same, with $e^{-i(k_x x + k_y y)}$

Also: linear; commutative, $f * g = g * f$; shift-equivariant, $f(\cdot - a) * g = (f * g)(\cdot - a)$.

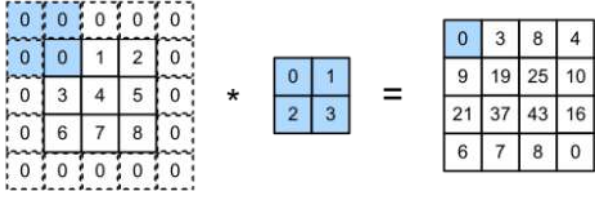
Discrete Convolution in 2D

Image $x(i,j)$, kernel $G(m,n)$ of size $k \times k$:

$$\text{convolution:} \quad (G * x)(i,j) = \sum_m \sum_n G(m,n)\, x(i-m,\, j-n)$$

$$\text{cross-correlation (what CNN layers compute):} \quad S(i,j) = \sum_m \sum_n G(m,n)\, x(i+m,\, j+n)$$

The two differ only by flipping the kernel. The kernel is *learned*, so the flip does not matter: deep-learning libraries compute the cross-correlation and still call it "convolution".

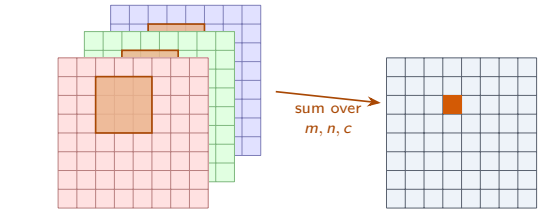


a 2×2 kernel slides over a zero-padded 3×3 input; each output is the sum of element-wise products, e.g.

$$0 \cdot 0 + 0 \cdot 1 + 0 \cdot 2 + 1 \cdot 3 = 3$$

Images with Three Channels

A colour image has three numbers per pixel, red, green and blue: $x(i, j, c)$ with $c \in \{R, G, B\}$. A CIFAR-10 image is $32 \times 32 \times 3$.



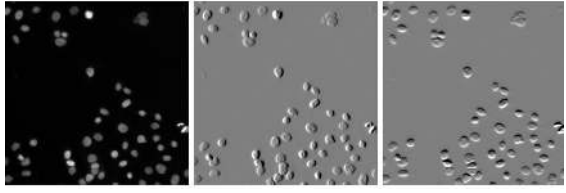
$$S(i, j \mid c) = \sum_{m,n} G(m, n, c)\, x(i + m,\, j + n,\, c), \qquad S(i, j) = \sum_{c} S(i, j \mid c) + b$$

each channel is convolved with its own slice of the kernel, and the three results are added: one kernel gives *one* feature map; C_{out} kernels give C_{out} feature maps, which are the channels of the next layer's input

Kernels Detect Features: Sobel Filters

$$\vec{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \qquad \vec{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

approximate the intensity gradient: they light up where the image changes sharply — **edges**



human cell nuclei, fluorescence microscope · vertical edges ($\vec{G}_x$) · horizontal edges ($\vec{G}_y$)

Sharpening, blurring and the identity are 3×3 kernels too.

Idea of a CNN: do not design the kernels, learn them.

image: D. Root, CC0, from J. Moffat et al., *A Lentiviral RNAi Library for Human and Mouse Genes Applied to an Arrayed Viral High-Content Screen*, Cell **124**, 1283 (2006), doi:10.1016/j.cell.2006.01.040

Kernels Are Learnable Parameters

-1	0	$+1$
-2	0	$+2$
-1	0	$+1$

Sobel $\vec{G}_x$:
fixed, designed by hand

G_{11}	G_{12}	G_{13}
G_{21}	G_{22}	G_{23}
G_{31}	G_{32}	G_{33}

CNN kernel:
9 trainable weights

- the entries $G(m, n, c)$ and the bias b are the **weights** of a convolutional layer, as W and b are for a dense layer
- they start random and are trained by gradient descent on the same loss:

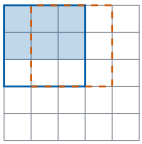
$$G(m, n, c) \leftarrow G(m, n, c) - \eta \frac{\partial \mathcal{L}}{\partial G(m, n, c)}$$

- backpropagation gives the gradient; one kernel is shared by all positions, so its gradient sums over all of them

After training, first-layer kernels often look like edge detectors.

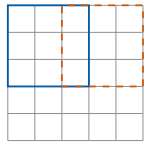
Kernel Size, Stride, Padding

stride 1



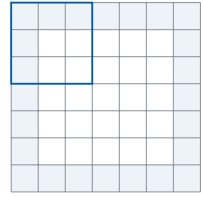
5×5 input, 3×3 kernel $\rightarrow 3 \times 3$ output

stride 2



the kernel jumps 2 pixels $\rightarrow 2 \times 2$ output

padding 1



zeros around the border $\rightarrow 5 \times 5$ output ("same")

Output size along one dimension

$$O = \left\lfloor \frac{W - k + 2P}{s} \right\rfloor + 1$$

W : input size, k : kernel size
P : padding, s : stride

CIFAR-10: $W = 32$, $k = 3$, $P = 1$, $s = 1 \Rightarrow O = 32$; with $s = 2$: $O = 16$

Pooling: Max, Average, Global Average

Window 2×2, stride 2: each 2×2 block of the feature map becomes *one* number.

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 4 & 2 & 1 & 3 \\ 9 & 8 & 6 & 5 \end{bmatrix}$$

$$\xrightarrow{\text{max pooling}} \begin{bmatrix} 6 & 8 \\ 9 & 6 \end{bmatrix}$$

$$\xrightarrow{\text{average pooling}} \begin{bmatrix} 3.75 & 5.25 \\ 5.75 & 3.75 \end{bmatrix}$$

$$\xrightarrow{\text{global average pooling}} 4.625$$

top-left block: $\max(1,3,5,6) = 6$, $\quad (1+3+5+6)/4 = 3.75$; $\quad$ global: $74/16 = 4.625$

- **max pooling**: the strongest response; used between conv layers
- **global average pooling**: one number per feature map; used at the end of a network
- no trainable parameters

Downsampling

4 times fewer numbers; the next 3×3 kernel covers a 6×6 region; small shifts do not change the output. The price: exact positions are lost.

CNNs Learn a Hierarchy of Features

- a CNN is a neural network that **learns the kernels** for each channel
- layers of kernels capture a **hierarchy of features**: edges $\rightarrow$ parts $\rightarrow$ objects



first layer | middle layer | deep layer

features learned from face images by a convolutional deep belief network: H. Lee, R. Grosse, R. Ranganath, A. Y. Ng, *Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations*, ICML 2009, doi:10.1145/1553374.1553453

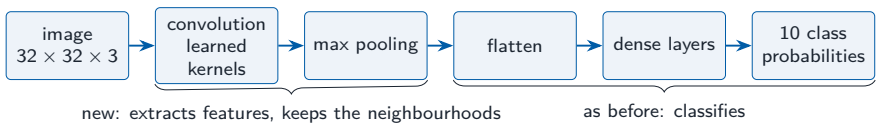
Second Attempt at CIFAR-10 Classification

First attempt: a dense network

- the image is flattened into 3072 numbers
- which pixels are neighbours is thrown away
- 46% training, 41% test accuracy

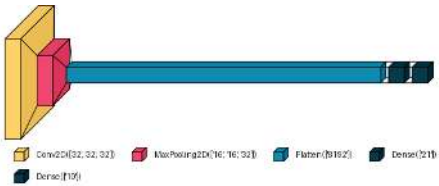
Second attempt: a convolutional network

- the image stays an image: $32 \times 32 \times 3$
- kernels look at small neighbourhoods and are *learned*
- pooling reduces the size of the feature maps

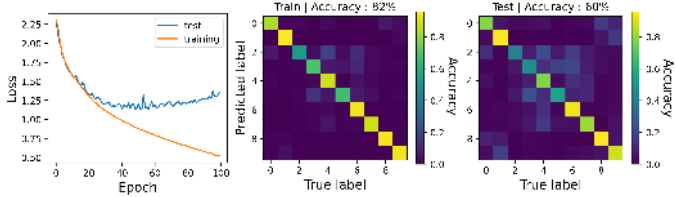


Same data, same loss (categorical cross-entropy), same training loop: **only the model changes.**

A Small CNN on CIFAR-10

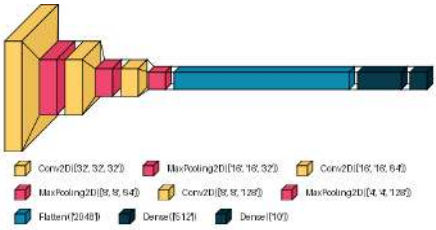


- Conv2D: 32 kernels 3×3, sigmoid, padding "same" $\rightarrow 32 \times 32 \times 32$
- MaxPooling $2 \times 2 \rightarrow 16 \times 16 \times 32$
- flatten $\rightarrow$ 8192, dense 21 (sigmoid), dense 10 (softmax)

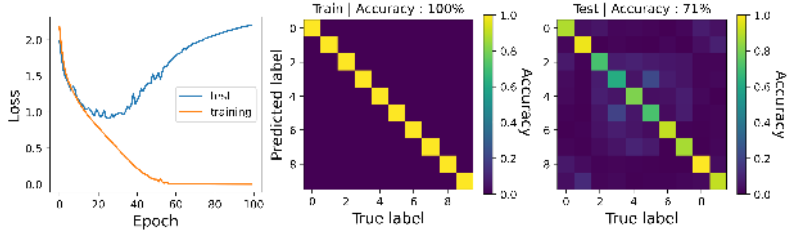


About 82% training, 60% test accuracy — better than 41%, but the test loss **rises after ∼40 epochs**: *overfitting*.

More CNN Layers: Still Overfitting



three blocks Conv2D (ReLU) + MaxPooling:
$32 \times 32 \times 32 \to 16 \times 16 \times 64 \to 8 \times 8 \times 128$,
then flatten, dense 512, dense 10



100% training, 71% test accuracy: **more complexity does not cure overfitting**.

2

Regularisation: Curing Overfitting

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Regularisation: Three Common Techniques

Regularisation: any technique that constrains training or model capacity so that the network *generalises* to unseen data instead of memorising the training set.

L1 / L2 penalty

a term in the loss that keeps the weights small

Dropout

randomly switch off neurons during training

Batch normalisation

normalise the inputs of each hidden layer over a mini-batch

Also widely used: *early stopping* (stop when the validation loss starts to rise), *data augmentation* (shifted, flipped, cropped copies of the training images), *learning-rate scheduling*.

L1 / L2 Regularisation

L1 (Lasso)

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i |\theta_i|$$

drives some weights exactly to zero: *sparse* solutions

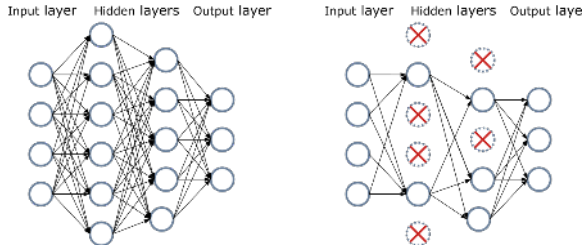
L2 (Ridge, "weight decay")

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{original}} + \lambda \sum_i \theta_i^2$$

discourages large weights: *smoother* models

- θ_i: the weights of the network; λ: the regularisation strength, a hyperparameter
- a **soft constraint**: weights are encouraged, not forced, to stay small — λ plays the role of a Lagrange multiplier
- $\lambda = 0$: no regularisation; λ too large: the network underfits

Dropout



- at every training step each neuron of the layer is switched off with probability p (the *dropout rate*, typically 0.2–0.5)
- the network cannot rely on any single neuron: it learns **redundant representations**
- each step trains a different sub-network: dropout acts as an **ensemble** of sub-networks
- at test time all neurons are active

N. Srivastava et al., *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, JMLR **15**, 1929 (2014), jmlr.org/papers/v15/srivastava14a.html

Batch Normalisation

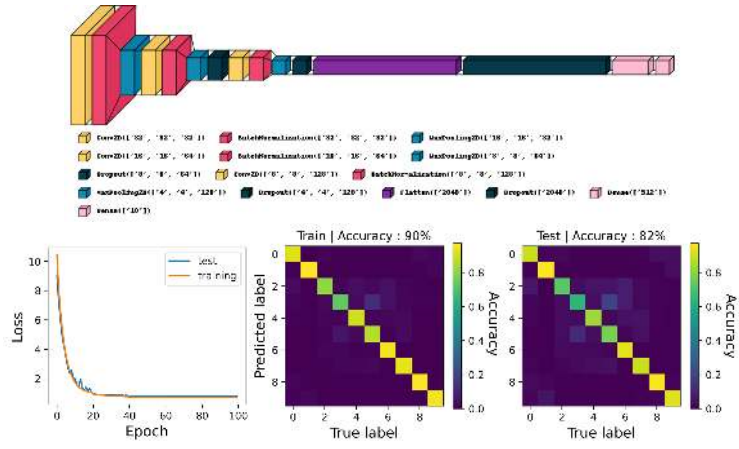
A BatchNorm layer normalises the input of a hidden layer over the mini-batch $\{\mathbf{x}_j\}_{j=1}^{N_B}$:

$$\mu_B = \frac{1}{N_B}\sum_{j=1}^{N_B}\mathbf{x}_j, \qquad \sigma_B^2 = \frac{1}{N_B}\sum_{j=1}^{N_B}(\mathbf{x}_j-\mu_B)^2 \qquad \mathbf{x}_j \leftarrow \frac{\mathbf{x}_j-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}, \qquad \mathbf{x}_j \leftarrow \gamma\,\mathbf{x}_j+\beta$$

- normalise: zero mean and unit variance within the mini-batch (ϵ: a small constant for numerical stability)
- then rescale and shift with **trainable** γ and β
- makes the network less dependent on weight initialisation
- reduces sensitivity to weight changes; helps against exploding and vanishing gradients

S. Ioffe, C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML 2015, arXiv:1502.03167

The Larger CNN with Regularisation



Same three Conv2D blocks, now with batch normalisation and dropout: 90% training, 82% test accuracy.

Before: 100% / 71%. **The training and test losses now stay together.**

3

Class Activation Maps: Where the CNN Looks

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Class Activation Maps

input (32 x 32)



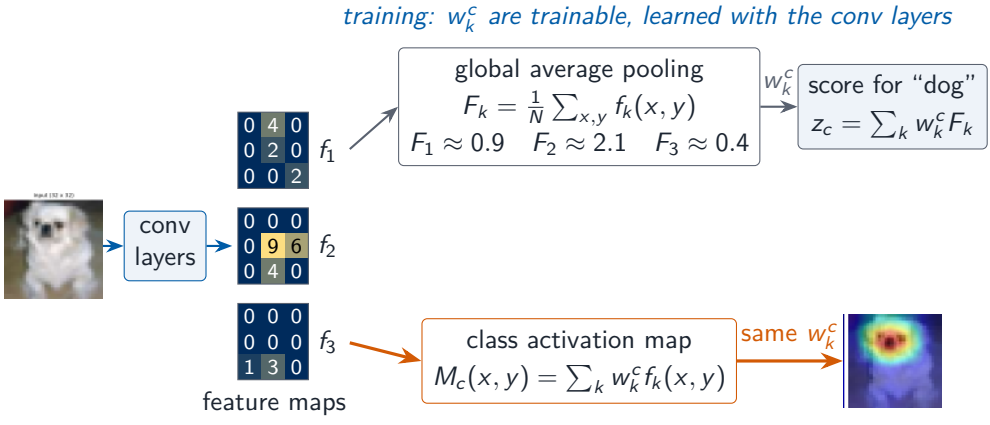
a CIFAR-10 test image; our CNN: "dog", 1.00

Which parts of the image were important for the prediction?

Can we get insight into what the CNN was looking at when it made its prediction?

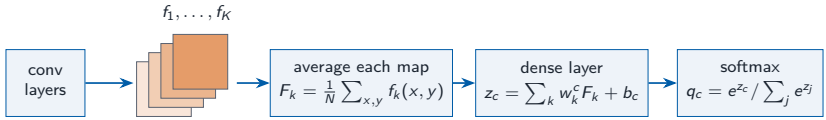
A **class activation map** (CAM) answers with a picture: a map over the image that shows *where* the evidence for a class is.

From Feature Maps to a Class Map



B. Zhou et al., *Learning Deep Features for Discriminative Localization*, CVPR 2016, arXiv:1512.04150

CAM: From the Class Score Back to a Map



K feature maps, each $h \times w$, $N = hw$ pixels

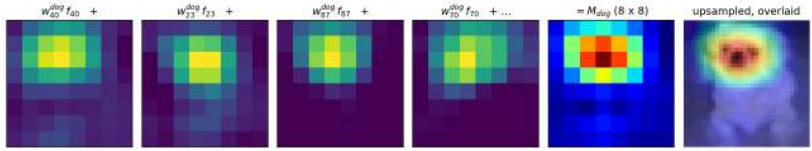
$$z_c = \sum_k w_k^c F_k + b_c = \sum_k w_k^c \frac{1}{N} \sum_{x,y} f_k(x,y) + b_c = \frac{1}{N} \sum_{x,y} \underbrace{\sum_k w_k^c f_k(x,y)}_{M_c(x,y)} + b_c$$

Class activation map

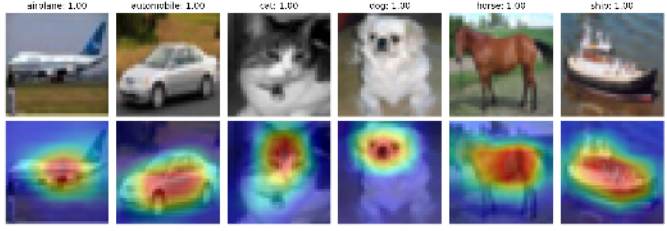
$$M_c(x,y) = \sum_k w_k^c f_k(x,y)$$

the class score z_c is the average of M_c over all positions, plus b_c

CAM on CIFAR-10



feature maps times their weights for "dog", added to one 8×8 map, enlarged and laid over the image



correctly classified test images: the evidence sits on the object, not on the background

our CNN with GAP: 6 conv layers $\rightarrow$ 128 maps of 8×8 $\rightarrow$ GAP $\rightarrow$ 10 outputs; 87.6% test accuracy

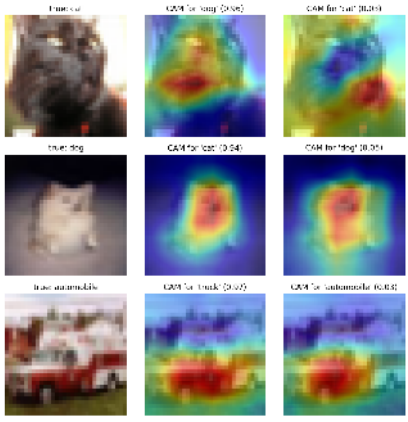
CAM for Two Classes in One Photograph



- the map of a dog class lies on the dog; the map of a cat class lies on the cat
- network: ResNet-18 trained on ImageNet (1000 classes), also ending with global average pooling
- p: softmax probability of that class. It is small: two small animals, 1000 classes to share the probability.

photograph: Socks and Buddy, U.S. National Archives, public domain, Wikimedia Commons

CAM: Mistakes and Limits



map of the predicted class (middle) and of the true class (right)

- **cat → "dog" (0.96):** a dark close-up; the "dog" evidence on the lower part of the face
- **dog → "cat"**, **automobile → "truck"**: both maps on the object, the wrong one of two similar classes

CAM tells us

where the evidence for a class is: on the object or on the background (shortcut learning)

CAM does not tell us

why that region mattered; fine detail (resolution of the last feature map, 8×8 here)

Accurate for the wrong reason: wolves told from huskies by the *snow* in the background. M. T. Ribeiro et al., *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, KDD 2016, arXiv:1602.04938

Grad-CAM: Class Activation Maps for Any CNN

- $f_k(x, y)$: feature maps of the last convolutional layer; z_c: score of class c before softmax
- CAM needs one weight w_k^c per map; when other layers follow the last convolution, Grad-CAM uses the gradient averaged over the N pixels: $\alpha_k^c = \dfrac{1}{N} \sum_{x,y} \dfrac{\partial z_c}{\partial f_k(x, y)}$

CAM

$$M_c(x, y) = \sum_k w_k^c \, f_k(x, y)$$

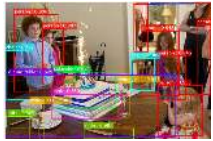
Grad-CAM

$$M_c(x, y) = \mathrm{ReLU}\Big(\sum_k \alpha_k^c \, f_k(x, y) \Big)$$

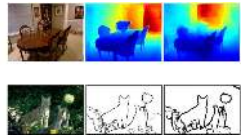
- ReLU keeps only the regions that *increase* z_c
- GAP network: $\alpha_k^c = w_k^c / N$, and Grad-CAM gives the CAM map

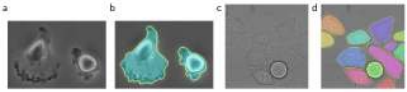
R. R. Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017, arXiv:1610.02391

Detection and Segmentation

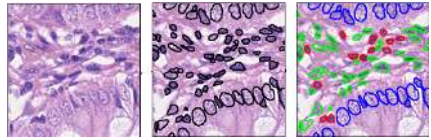








living cells: input (a, c), one colour per cell (b, d)



tissue: input, nuclei outlined, coloured by type

- **detection:** a box and a class for every object
- **segmentation:** a class for every pixel, and in cells and tissues every object separately: counts, sizes, shapes

O. Ronneberger, P. Fischer, T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI 2015, arXiv:1505.04597; S. Graham et al., *HoVer-Net: Simultaneous Segmentation and Classification of Nuclei in Multi-Tissue Histology Images*, Med. Image Anal. **58**, 101563 (2019), doi:10.1016/j.media.2019.101563

Where We Are

data	model	train	test
MNIST	dense, 1 output, MSE	58%	56%
MNIST	dense, 10 outputs, cross-entropy	94%	95%
CIFAR-10	dense, 10 outputs, cross-entropy	46%	41%
CIFAR-10	small CNN	82%	60%
CIFAR-10	larger CNN	100%	71%
CIFAR-10	larger CNN with batch norm, dropout	90%	82%
CIFAR-10	CNN with GAP, batch norm, augmentation	91%	88%

first six rows: book Ch. 2; last row: our CAM network

Regularisation closes the gap between training and test.

4

When Networks Fail: Adversarial Examples

JAGIELLONIAN UNIVERSITY
IN KRAKÓW

Adversarial Examples



- **left:** a photograph. The trained classifier says "panda", with 57.7% confidence — correct.
- **middle:** a pattern computed from the network. Shown alone it means nothing: the classifier's best guess is "nematode" (a roundworm), with 8.2%.
- **right:** photograph + 0.007× pattern. For us the same photograph; the classifier says "gibbon", with 99.3% confidence.

Such inputs are called **adversarial examples**.

I. J. Goodfellow, J. Shlens, C. Szegedy, *Explaining and Harnessing Adversarial Examples*, ICLR 2015, arXiv:1412.6572

One-Pixel Attack

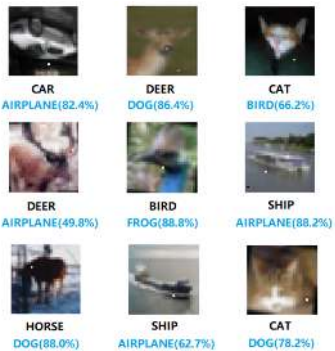


Fig. 1: CIFAR-10



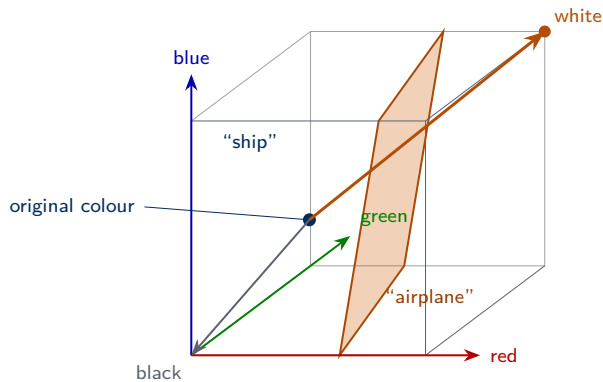
Fig. 2: ImageNet, pixel in the red circle

- black: the true class; blue: the class after changing one pixel, with the network's confidence
- Kaggle CIFAR-10 test set: 67.97% of the images can be moved to at least one other class by one pixel, with 74.03% confidence on average
- ImageNet: 16.04% of the test images, 22.91% confidence

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack: Mechanism

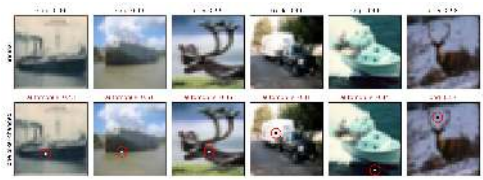
- an image is one point in a space with 3072 axes
- the network cuts this space into class regions, separated by **decision boundaries**
- one pixel = 3 of these axes: its red, green and blue value
- if a boundary is close along these 3 axes, changing the colour of that pixel moves the image into another class



the cube of all colours of *one* pixel; orange: the decision boundary. To white: the class changes. To black: it does not.

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

One-Pixel Attack: Finding the Pixel



The search in the paper

- 1024 positions $\times$ 256^3 colours: too many to try all
- a candidate: five numbers (x, y, R, G, B)
- its quality: the probability of the true class — the lower, the better
- differential evolution: 400 random candidates, mix them, keep the better ones, repeat
- black box: no gradients, only output probabilities

On our CIFAR-10 network

- the CNN of the CAM section (87.6% test accuracy)
- plain random search: 1000 candidates per image, extreme colours (black, white, ...)
- 111 of 300 correctly classified images change class (37.0%); average confidence in the wrong class 71%
- top: original; bottom: one pixel changed (red circle)

J. Su, D. V. Vargas, K. Sakurai, *One Pixel Attack for Fooling Deep Neural Networks*, IEEE Trans. Evol. Comput. **23**, 828 (2019), doi:10.1109/TEVC.2019.2890858, arXiv:1710.08864

Adversarial T-Shirts



Video frames with two people. The detector (YOLOv2) draws a box around each person it finds; the person wearing the printed pattern gets no box. Attack success rate: 74% in the digital world, 57% with a printed T-shirt on a moving person.

K. Xu et al., *Adversarial T-shirt! Evading Person Detectors in A Physical World*, ECCV 2020, arXiv:1910.11099

Attacks on Deep Networks: Inputs and Training

attack	what the attacker does	reference
at test time: the input is manipulated		
adversarial example	adds a small, designed perturbation to one input	*Intriguing properties of neural networks*, Szegedy et al. 2014
universal perturbation	one pattern that fools the network on most images	*Universal adversarial perturbations*, Moosavi-Dezfooli et al. 2017
adversarial patch	a printed sticker placed anywhere in the scene	*Adversarial Patch*, Brown et al. 2017
at training time: the data or the training are manipulated		
data poisoning	injects crafted samples into the training set	*Poisoning Attacks against Support Vector Machines*, Biggio et al. 2012
backdoor	hides a trigger: the model is normal until the trigger appears	*BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain*, Gu et al. 2017
attack on federated learning	one participant of a joint training plants a backdoor	*How To Backdoor Federated Learning*, Bagdasaryan et al. 2018

Attacks on Deep Networks: Privacy and Language Models

attack	what the attacker does	reference
against privacy and ownership: information is extracted		
model stealing	rebuilds the model from its answers to queries	*Stealing Machine Learning Models via Prediction APIs*, Tramèr et al. 2016
membership inference	finds out whether a given record was in the training set	*Membership Inference Attacks against Machine Learning Models*, Shokri et al. 2017
model inversion	reconstructs training data from the model's outputs	*Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures*, Fredrikson et al. 2015
language models		
prompt injection	hides instructions in the text or web page the model reads	*Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*, Greshake et al. 2023

Medical records, patient images, unpublished measurements: the privacy attacks concern scientific data directly.

Limits of Deep Networks

About the networks

- high test accuracy says nothing about inputs *outside* the distribution of the training data
- the network does not use the features we use: it uses whatever separates the training classes
- confidence is not correctness: the wrong answers come with high probability

In scientific data, without any attacker

- a new microscope, a different staining, another detector or hospital: the inputs shift, the accuracy can drop
- a network can learn a shortcut: a marker, a scale bar, the background, instead of the object
- tools to check: class activation maps, test data from an *independent* source

5

CNNs on Scientific Data: Blood Cells and Satellite Images



JAGIELLONIAN UNIVERSITY
IN KRAKOW

MedMNIST

MNIST

Modified National Institute of Standards and Technology database: 70 000 grey images, 28×28 pixels, of hand-written digits $0 \ldots 9$ — our "Hello world".

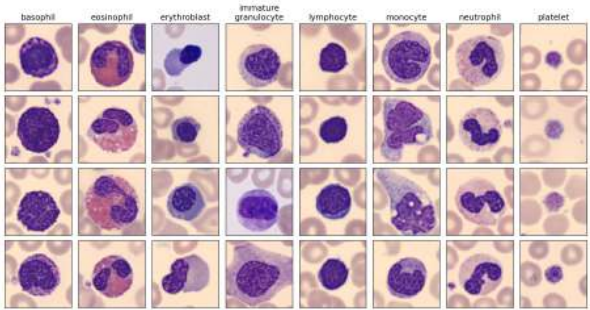
MedMNIST

the same idea for biomedical images: small images of a fixed size, a fixed split into training, validation and test part, one file per dataset.

- MedMNIST v2: 12 datasets of 2D images and 6 datasets of 3D images
- pathology slides, chest X-rays, skin lesions, retina scans, **blood cells**, organs in CT, ...
- images at 28×28 pixels; larger versions at 64, 128 and 224 pixels
- one line of code to load: `pip install medmnist`

J. Yang et al., *MedMNIST v2 — A large-scale lightweight benchmark for 2D and 3D biomedical image classification*, Sci. Data **10**, 41 (2023), doi:10.1038/s41597-022-01721-8; medmnist.com

BloodMNIST: Blood Cells under the Microscope



- 17 092 microscope images of single blood cells
- colour images, 64×64 pixels
- label: one of 8 cell types, assigned by clinical pathologists

The task:
image of one cell $\rightarrow$ its type, e.g. **"neutrophil"**

A. Acevedo et al., *A dataset of microscopic peripheral blood cell images for development of automatic recognition systems*, Data in Brief **30**, 105474 (2020), doi:10.1016/j.dib.2020.105474

The Eight Cell Types

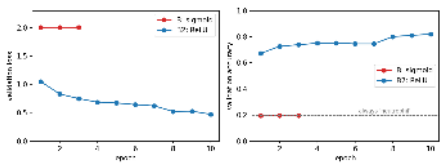
label		cell type	what it is	images
0		basophil	granulocyte with large dark granules	852
1		eosinophil	granulocyte with a two-lobed nucleus	2 181
2		erythroblast	immature red blood cell with a nucleus	1 085
3		immature granulocyte	precursor of granulocytes	2 026
4		lymphocyte	small cell with a large round nucleus	849
5		monocyte	largest white blood cell, folded nucleus	993
6		neutrophil	most common white blood cell, lobed nucleus	2 330
7		platelet	small cell fragment without a nucleus	1 643

The classes are **not balanced**: always answering "neutrophil" is right for 19.5% of the training images.

Blood Cells: the Models and Their Accuracy

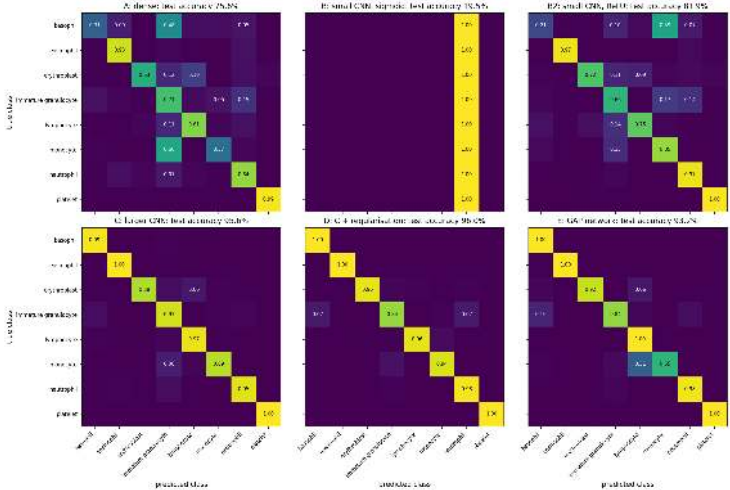
	model	parameters	train	val.	test
A	dense network on the flattened image	258 245	78.3%	75.2%	75.6%
B	small CNN, sigmoid	689 221	19.5%	19.5%	**19.5%**
B2	the same small CNN, ReLU	689 221	86.0%	82.0%	81.9%
C	larger CNN	4 292 168	99.2%	95.5%	95.6%
D	C + batch norm, dropout, L2, augmentation	4 292 616	96.2%	96.4%	96.0%
E	GAP network	288 936	93.5%	94.0%	93.2%

same data, loss and optimiser; 10 epochs each



- Model B answers "neutrophil" for every image: 19.5%, the share of neutrophils. Its loss, 2.003, is the entropy of the class frequencies.
- its sigmoids are saturated: the spread of their inputs grew from 0.35 to 98.60, and the gradient vanishes
- the same network with ReLU: **81.9%**

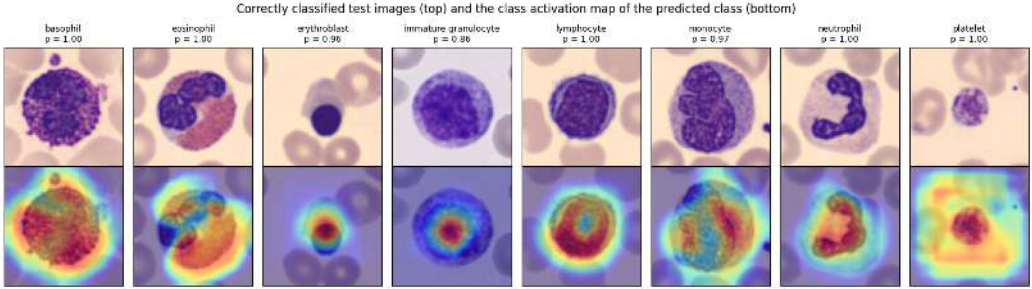
Blood Cells: Confusion Matrices of All Models



rows: true class; columns: predicted class

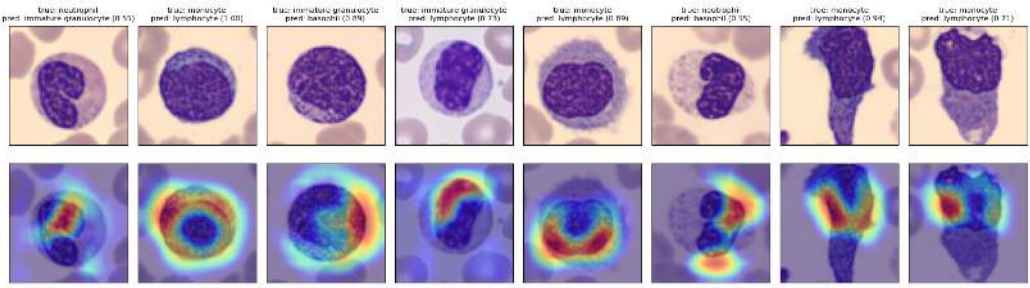
- A: 60% of monocytes taken for immature granulocytes
- B: one column, “neutrophil” for every image
- B2: 59% of basophils taken for monocytes
- C, D: almost diagonal
- E: 31% of monocytes taken for lymphocytes

CAM on Blood Cells



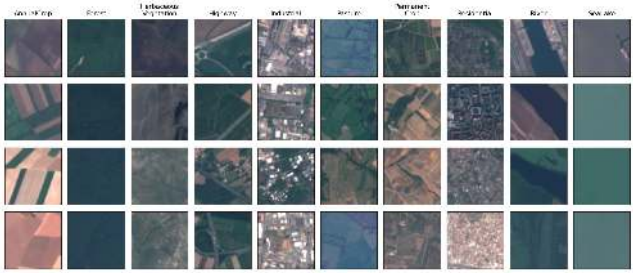
- top row: correctly classified test images, one per class
- bottom row: class activation map of the predicted class
- the evidence lies on the cell itself
- not on the red blood cells around it

Blood Cells: the Mistakes



- 233 of 3 421 test images are misclassified
- eight of them shown, with the map of the *predicted* class
- largest confusion: 31% of the monocytes are taken for lymphocytes
- an earlier run confused a different pair: errors are not stable from run to run

EuroSAT: Satellite Images of the Earth's Surface



- 27 000 patches from the Sentinel-2 satellites
- 64×64 pixels: 640 m $\times$ 640 m on the ground
- label: one of 10 land-use classes

The task:
one patch $\rightarrow$ its land-use class, e.g. **"River"**

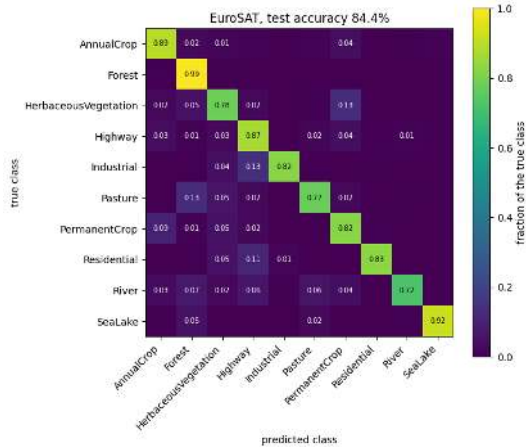
P. Helber et al., *EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification*, IEEE JSTARS **12**, 2217 (2019), doi:10.1109/JSTARS.2019.2918242

EuroSAT: Ten Land-Use Classes

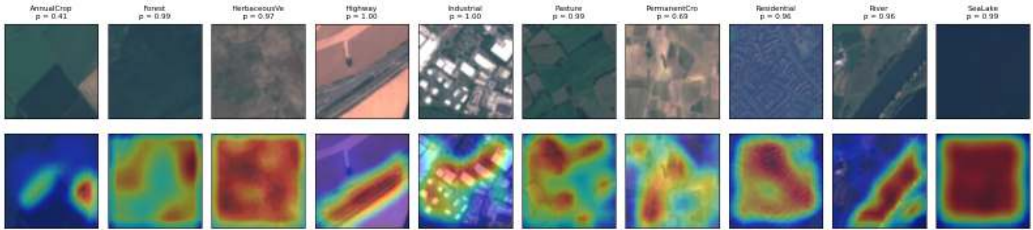
	class	on the ground	images
0	AnnualCrop	fields sown and harvested every year: wheat, maize	3 000
1	Forest	forest	3 000
2	HerbaceousVegetation	grass and other low natural vegetation	3 000
3	Highway	motorway or major road	2 500
4	Industrial	industrial buildings	2 500
5	Pasture	grassland for grazing animals	2 000
6	PermanentCrop	orchards, vineyards	2 500
7	Residential	residential buildings	3 000
8	River	a river crossing the patch	2 500
9	SeaLake	sea or lake	3 000

EuroSAT: Results

- the GAP network of the blood cells, with 10 outputs
- augmentation: flips, shifts, rotations by 90°
- 1 000 patches per class, 10 epochs: **84.4% test accuracy**
- best: Forest (99.5%); worst: River (72.2%)
- an earlier run of the same code: 90.0%



CAM on EuroSAT



- top row: one correctly classified test patch per class
- bottom row: class activation map of the predicted class
- highway, river: the map follows the object
- forest, sea: the map covers the whole patch

6

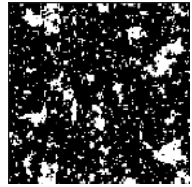
A Physics Example: the 2D Ising Model



JAGIELLONIAN UNIVERSITY
IN KRAKOW

The Ising Model in Plain Words

- a grid of cells, each in one of **two states** (spin up or down)
- each cell **prefers the state of its neighbours**
- **temperature** is random noise: it flips cells whatever their neighbours do
- low noise: one state spreads over the whole grid; high noise: a random mosaic. In between lies a sharp threshold, the **critical temperature** T_c



a sampled configuration at T_c: patches of all sizes

The same mathematics elsewhere

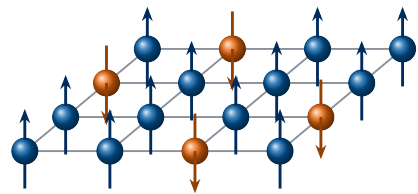
- **magnets:** iron is magnetic below its Curie temperature, $1043\,\mathrm{K}$ ($770\,^{\circ}\mathrm{C}$), and not above it
- **neurons:** the joint firing patterns of retinal neurons (active or silent) are described by an Ising model fitted to recordings
- **images:** neighbouring pixels of a map or a satellite image tend to share a class; Ising-type models are used as priors to restore and segment such images

Our question: can a network find T_c only by looking at snapshots of the grid, as it classified blood cells and land use?

E. Schneidman, M. J. Berry, R. Segev, W. Bialek, *Weak pairwise correlations imply strongly correlated network states in a neural population*, Nature **440**, 1007 (2006), doi:10.1038/nature04701; S. Geman, D. Geman, *Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images*, IEEE TPAMI **6**, 721 (1984), doi:10.1109/TPAMI.1984.4767596

The 2D Ising Model

- a square lattice $L \times L$; on every site i a *spin* $s_i = +1$ (up) or $s_i = -1$ (down)
- a simple model of a magnet: each spin is a tiny magnetic moment
- $2^{L \cdot L}$ possible configurations $\{s_i\}$



Energy of a configuration (Hamiltonian)

$$H(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$$

$\langle i, j \rangle$: pairs of nearest neighbours; $J > 0$: aligned neighbours lower the energy (ferromagnet), we set $J = 1$; h: external magnetic field, we set $h = 0$.

The Couplings Decide the Lowest-Energy State

$H = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$: the sign of J decides which neighbours are favourable.
Lowest-energy configurations at $h = 0$:



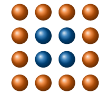
$J > 0$: ferromagnetic

neighbours *aligned*
all up or all down: 2 ground states
energy $-2J$ per spin
magnetisation $M = 1$



$J < 0$: antiferromagnetic

neighbours *opposite*
checkerboard pattern: 2 ground states
energy $-2|J|$ per spin
magnetisation $M = 0$



$J = 0$: non-interacting

spins independent
all $2^{L \cdot L}$ configurations have the same energy
no order at any temperature

- square lattice: each spin has 4 neighbours, so 2 bonds per spin
- $h = 0$: flipping *all* spins leaves H unchanged — hence two equivalent ground states
- $h > 0$: spins prefer $+1$; $h < 0$: spins prefer -1; the field removes this symmetry and selects one state

blue: $s_i = +1$, orange: $s_i = -1$. From here on: $J = 1$ (ferromagnet), $h = 0$.

The Partition Function

At temperature T ($k_B = 1$) every configuration appears with the Boltzmann probability

$$p(\{s_i\}) = \frac{e^{-H(\{s_i\})/T}}{Z}, \qquad Z(T) = \sum_{\{s_i\}} e^{-H(\{s_i\})/T}$$

Z, the **partition function**, is the normalisation: a sum over all $2^{L \cdot L}$ configurations.

Knowing Z gives the physical properties as functions of T ($\beta = 1/T$)		
mean energy	$\langle H \rangle = -\dfrac{\partial \ln Z}{\partial \beta}$	average energy of the configurations at T
specific heat	$C = \dfrac{d\langle H \rangle}{dT} = \beta^2 \dfrac{\partial^2 \ln Z}{\partial \beta^2}$	how fast the energy grows with T; proportional to the energy fluctuations
mean magnetisation	$\langle S \rangle = \dfrac{1}{\beta} \dfrac{\partial \ln Z}{\partial h}, \quad S = \sum_i s_i$	net alignment of the spins: $\neq 0$ ordered, 0 disordered
susceptibility	$\chi = \dfrac{d\langle S \rangle}{dh} = \dfrac{1}{\beta} \dfrac{\partial^2 \ln Z}{\partial h^2}$	how strongly the spins follow a small field h; proportional to the magnetisation fluctuations

Solving the model means finding Z, or $\ln Z$, in analytical form as a function of (J, h, T); all equilibrium properties then follow by differentiation.

Onsager's Solution of the Partition Function

Every observable is a derivative of $\ln Z$, but $Z = \sum_{\{s_i\}} e^{-\beta H(\{s_i\})}$ is a sum over 2^N ($N = L^2$ spins) configurations. **Computing $\ln Z$ in closed form solves the model.**

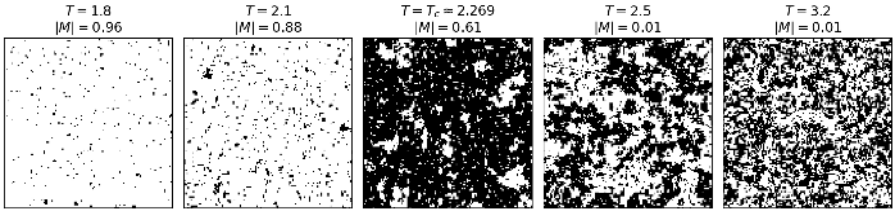
Onsager (1944): $\ln Z$ computed exactly for the 2D square lattice, $h = 0$, $N \to \infty$

$$\frac{1}{N}\ln Z = \ln\!\left(2\cosh(2\beta J)\right) + \frac{1}{2\pi}\int_0^{\pi} d\phi\, \ln\!\left[\tfrac{1}{2}\left(1+\sqrt{1-\kappa^2\sin^2\phi}\right)\right], \qquad \kappa = \frac{2\sinh(2\beta J)}{\cosh^2(2\beta J)}$$

- **method:** write $Z = \mathrm{Tr}\, V^L$, with V the $2^L \times 2^L$ *transfer matrix* that adds one row of spins; for a large lattice $\ln Z$ is fixed by the largest eigenvalue of V, and Onsager found all eigenvalues exactly
- $\kappa = 1$ exactly when $\sinh(2J/T) = 1$: there $\ln Z$ is **not analytic**, so $T_c = \dfrac{2J}{\ln(1+\sqrt{2})} \approx 2.269\, J$ (located already in 1941 by Kramers and Wannier from a low/high-temperature duality)
- near T_c: $C \sim -\ln|T - T_c|$, $\langle M\rangle \sim (T_c - T)^{1/8}$, $\chi \sim |T - T_c|^{-7/4}$, $\xi \sim |T - T_c|^{-1}$: a continuous transition with exponents $\alpha = 0$, $\beta = \frac{1}{8}$, $\gamma = \frac{7}{4}$, $\nu = 1$
- $\langle M\rangle = [1 - \sinh^{-4}(2J/T)]^{1/8}$ was announced by Onsager in 1949 and derived by C. N. Yang in 1952

H. A. Kramers, G. H. Wannier, *Statistics of the Two-Dimensional Ferromagnet. Part I*, Phys. Rev. **60**, 252 (1941), doi:10.1103/PhysRev.60.252; L. Onsager, *Crystal Statistics. I. A Two-Dimensional Model with an Order-Disorder Transition*, Phys. Rev. **65**, 117 (1944), doi:10.1103/PhysRev.65.117; C. N. Yang, *The Spontaneous Magnetization of a Two-Dimensional Ising Model*, Phys. Rev. **85**, 808 (1952), doi:10.1103/PhysRev.85.808

The Phase Transition in Pictures



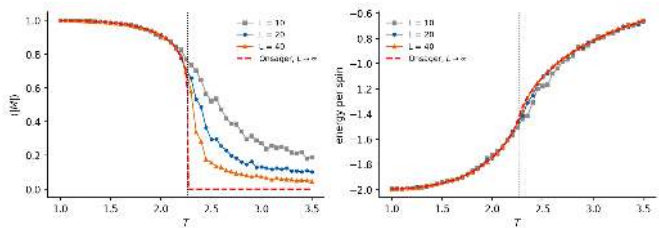
one sampled configuration at each temperature, 128×128 lattice ($N = 128^2$ spins); white and black: $s_i = +1$ and $s_i = -1$; $|M| = |\frac{1}{N} \sum_i s_i|$

- **below T_c:** the coupling between neighbours wins over thermal noise; almost all spins point the same way, up or down, and the lattice is a magnet by itself
- **above T_c:** thermal noise wins; small patches point randomly, $M = 0$
- **at T_c:** clusters of all sizes, from single spins to the size of the lattice: *critical behaviour*

In an infinite lattice M drops to zero exactly at T_c: a sharp change of the whole system caused by a small change of temperature.

A Phase Transition at a Critical Temperature

- **1D** (Ising, 1925): no phase transition at any $T > 0$
- **2D**: a phase transition at $T_c \approx 2.269$
- $T < T_c$: spontaneous magnetisation, $\langle M \rangle > 0$ without any external field
- $T > T_c$: $\langle M \rangle = 0$

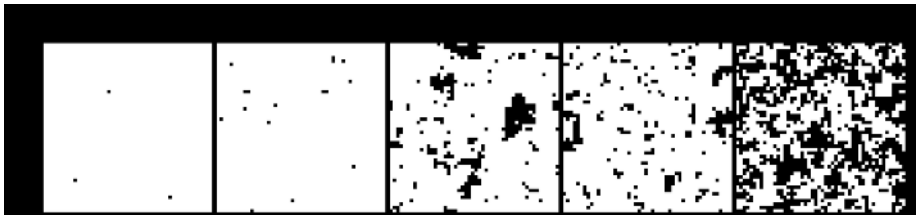


Monte Carlo averages for $L = 10, 20, 40$ (our data);
dashed red: Onsager's solution ($L \to \infty$); dotted: T_c

M is the **order parameter**: a quantity that tells the two phases apart. Near T_c: $M \sim (T_c - T)^{1/8}$, and fluctuations of observables become large.

Sampling Spin Configurations

$2^{L \cdot L}$ configurations are too many to list: Metropolis Monte Carlo draws samples from $p(\{s_i\}) \propto e^{-H/T}$ instead.

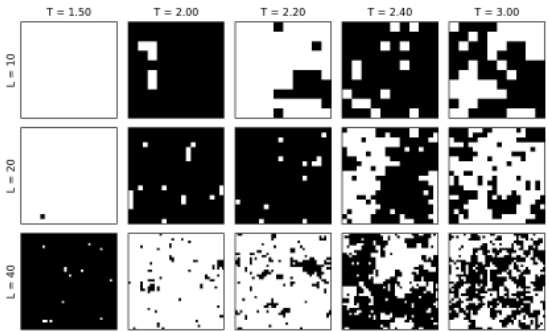


50×50 lattice at $T = 1.2,\ 1.9,\ 2.2,\ 2.4,\ 3.2$ (black and white: the two spin values)

Low T: spins aligned. High T: a random pattern.

N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, E. Teller, *Equation of State Calculations by Fast Computing Machines*, J. Chem. Phys. **21**, 1087 (1953), doi:10.1063/1.1699114

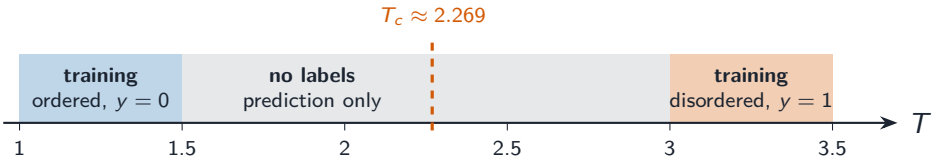
The Dataset



- Metropolis samples: 51 temperatures from 1.0 to 3.5, 200 configurations each, for $L = 10, 20, 40$
- each configuration is a black-and-white $L \times L$ image
- low T: almost all spins aligned; high T: random pattern
- on a small lattice $T = 2.2$ and $T = 2.4$ look alike, but lie on **opposite sides** of $T_c \approx 2.269$

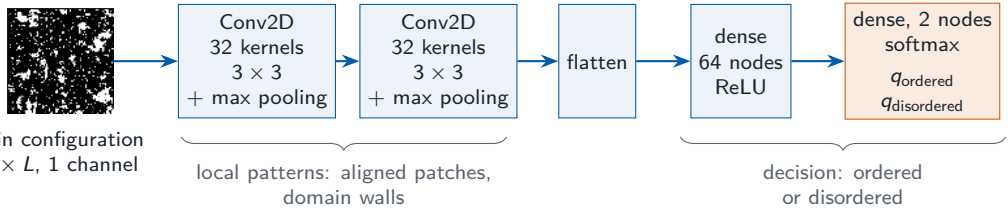
Task: find T_c from the images alone, without being told about the magnetisation.

Supervised Learning Far from the Transition



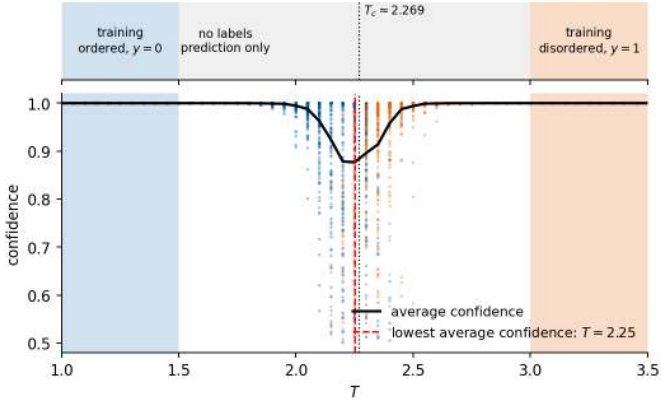
- **train only where the answer is clear:** $T < 1.5$ ordered, $T > 3$ disordered; configurations in between are left out
- **data:** input X = a spin configuration (an $L \times L$ image), label y = phase
- **predict** at every temperature, also where the network has seen no data, and plot the **confidence**: the softmax probability of the predicted class

The CNN for the Ising Model



- the same building blocks as for CIFAR-10: a spin configuration is a black-and-white image
- two outputs, softmax, categorical cross-entropy, Adam optimiser
- $q_{\text{ordered}} + q_{\text{disordered}} = 1$; the **confidence** is the larger of the two

The Confidence Drops at the Critical Temperature



$L = 40$; dots: the confidence of every configuration; line: its average. Lowest average confidence at $T = 2.25$; over three trainings 2.27 ± 0.02, exact $T_c \approx 2.269$.

Near T_c the magnetisation vanishes and fluctuations are large: the two phases are hardest to tell apart.

A First Discovery with a Neural Network

What we gained

- an estimate of T_c from raw configurations, with no formula for the order parameter
- for most physical systems the order parameter is *not known*: the network signals that *something happens* near a given temperature

Limits

- finite lattice: the transition is smoothed, the dip is broadened; larger L gives a sharper estimate
- the labels and the two training windows came from us
- the network does not tell us *why* there is a transition

The network located the phase transition; the physics that explains it is still ours to find.

The Chinese Room Thought Experiment

- a person who does not understand Chinese sits inside a room
- a rulebook (an algorithm) says how to manipulate Chinese characters to answer questions written in Chinese
- to an observer outside, the room understands Chinese: the answers are coherent
- the person understands nothing; they only follow rules



Searle: manipulating symbols (syntax) is not enough for understanding (semantics).

J. R. Searle, *Minds, brains, and programs*, Behav. Brain Sci. **3**, 417 (1980), doi:10.1017/S0140525X00005756

The Chinese Room: Replies

Searle's paper lists replies to the argument and answers them.

- **systems reply:** the person does not understand, but the whole system (person, rulebook, paper) does.
  Searle: let the person memorise the rulebook; they still understand nothing.

- **robot reply:** connect the symbols to the world through cameras and motors, and they acquire meaning.
  Searle: the sensor data are just more symbols for the person in the room.

- **brain simulator reply:** simulate the neurons of a Chinese speaker instead of following a rulebook.
  Searle: simulating the formal structure of the brain still does not give its understanding.

The brain simulator reply is close to what a neural network is; the debate is still open.

J. R. Searle, *Minds, brains, and programs*, Behav. Brain Sci. **3**, 417 (1980), doi:10.1017/S0140525X00005756

The Chinese Room and Deep Learning

- **data manipulation:** a network maps inputs to outputs by learned transformations, as the person in the room follows rules
- **statistical, not semantic:** the network's knowledge is a set of correlations in the data; it does not know what the pixels, words or spins mean
- **subsymbolic:** the room uses explicit rules; a network uses weights and activations learned from data

Back to the Ising model

The CNN located T_c without an explanation: no order parameter, no mechanism of the transition. The understanding (Onsager's solution, the order parameter) remains the scientist's job.

Summary

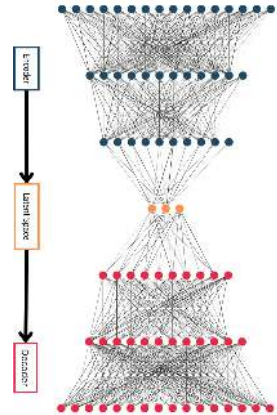
- **convolution**: small shared kernels see local patterns; CIFAR-10 test accuracy from 41% (dense network) to 88% (CNN with batch normalisation and augmentation)
- **regularisation**: L1/L2, dropout, batch normalisation, augmentation close the gap between training and test
- **class activation maps** show where the evidence for a class lies; Grad-CAM extends them to any CNN
- **adversarial examples**: a change invisible to us can flip the prediction
- **blood cells and satellite images**: models with similar accuracy make different mistakes; the confusion matrix and the class activation map show which
- **2D Ising model**: a CNN trained far from the transition locates T_c from spin configurations alone; explaining the transition is still our job (the Chinese room)

Next Lecture: Unsupervised Learning

Most scientific data come without labels: microscope archives, sky surveys, simulations, sequencing data.

Unsupervised learning finds structure in data without labels:

- **dimensionality reduction**: PCA, t-SNE, UMAP
- **clustering**: groups of similar samples, e.g. cell types
- **autoencoders**: compress the data to a few latent variables and reconstruct it
- **anomaly detection**: samples the model cannot reconstruct



Example: the phases of the Ising model found from spin configurations alone, with no temperature labels. L. Wang, *Discovering phase transitions with unsupervised learning*, Phys. Rev. B **94**, 195105 (2016), doi:10.1103/PhysRevB.94.195105

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



